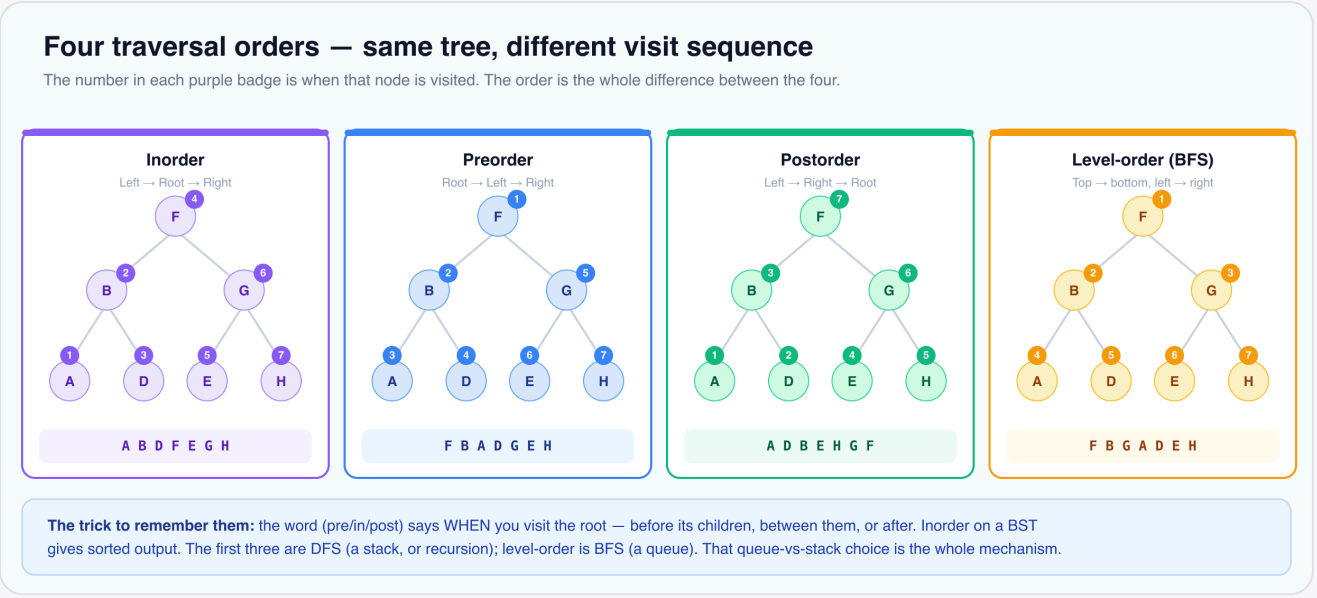
Trees

5 min read · Updated 16 Jul 2026 · Topic 65 of 77 · Bookmark · Mark done

23H. Tree Algorithms (20 problems)

Pattern recognition: Hierarchical data, parent-child, DOM-like structures.



23.91 Inorder Traversal (Iterative)

JAVASCRIPT · Copy

```javascript
function inorderTraversal(root) {
  const result = [], stack = [];
  let current = root;
  while (current || stack.length) {
    while (current) { stack.push(current); current = current.left; }
    current = stack.pop();
    result.push(current.val);
```

```
    current = current.right;
  return result;
}
```

23.92 Preorder Traversal

Approach: Iterative traversal using an explicit stack: pop a node, record it, then push its right child before its left so the left is processed first.

JAVASCRIPT Copy

```
function preorderTraversal(root){
  const res=[], stack=[]; if(root) stack.push(root);
  while(stack.length){
    const node=stack.pop(); res.push(node.val);
    if(node.right) stack.push(node.right);
    if(node.left) stack.push(node.left);
  }
  return res;
}
```

23.93 Postorder Traversal

Approach: Run a modified preorder (root, right, left) with a stack and unshift each value to the front, producing left-right-root order in reverse.

JAVASCRIPT Copy

```
function postorderTraversal(root){
  const res=[], stack=[]; if(root) stack.push(root);
  while(stack.length){
    const node=stack.pop(); res.unshift(node.val);
    if(node.left) stack.push(node.left);
    if(node.right) stack.push(node.right);
  }
  return res;
}
```

JAVASCRIPT Copy

```
function levelOrder(root) {
  if (!root) return [];
  const result = [], queue = [root];
  while (queue.length) {
    const level = [], size = queue.length;
    for (let i = 0; i < size; i++) {
      const node = queue.shift();
      level.push(node.val);
      if (node.left) queue.push(node.left);
      if (node.right) queue.push(node.right);
    }
    result.push(level);
  }
  return result;
}
```

23.95 Maximum Depth

JAVASCRIPT Copy

```
function maxDepth(root) {
  if (!root) return 0;
  return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

23.96 Same Tree

Approach: Recursively compare nodes: both null is equal, one null or differing values is unequal, otherwise recurse on matching left and right subtrees.

JAVASCRIPT Copy

```
function isSameTree(p,q){
  if(!p&&!q) return true;
  if(!p||!q||p.val≠q.val) return false;
```

23.97 Symmetric Tree

Approach: Mirror-compare the two subtrees, pairing each left child against the opposite right child, so a node is symmetric when its outer and inner reflections match.

JAVASCRIPT Copy

```
function isSymmetric(root){
  const mirror=(a,b)=>{
    if(!a&&!b) return true;
    if(!a||!b||a.val!==b.val) return false;
    return mirror(a.left,b.right)&&mirror(a.right,b.left);
  };
  return !root||mirror(root.left,root.right);
}
```

23.98 Invert Binary Tree

JAVASCRIPT Copy

```
function invertTree(root) {
  if (!root) return null;
  [root.left, root.right] = [invertTree(root.right), invertTree(root.left)
  return root;
}
```

23.99 Lowest Common Ancestor

JAVASCRIPT Copy

```
function lowestCommonAncestor(root, p, q) {
  if (!root || root === p || root === q) return root;
  const left = lowestCommonAncestor(root.left, p, q);
  const right = lowestCommonAncestor(root.right, p, q);
```

23.100 Validate BST

JAVASCRIPT Copy

```
function isValidBST(root, min = -Infinity, max = Infinity) {
  if (!root) return true;
  if (root.val ≤ min || root.val ≥ max) return false;
  return isValidBST(root.left, min, root.val) && isValidBST(root.right, ro
}
```

23.101 Kth Smallest in BST

Approach: Iterative inorder traversal with a stack visits nodes in ascending order; return the value on the kth pop.

JAVASCRIPT Copy

```
function kthSmallest(root,k){
  const stack=[]; let node=root;
  while(node||stack.length){
    while(node){ stack.push(node); node=node.left; }
    node=stack.pop();
    if(--k===0) return node.val;
    node=node.right;
  }
  return -1;
}
```

23.102 Binary Tree Right Side View

Approach: Level-order BFS by full levels, recording the last node of each level as the value visible from the right.

```
function rightSideView(root){
  const res=[]; if(!root) return res;
  let queue=[root];
  while(queue.length){
    res.push(queue[queue.length-1].val);
    const next=[];
    for(const n of queue){ if(n.left) next.push(n.left); if(n.right) next.
    queue=next;
  }
  return res;
}
```

23.103 Path Sum

Approach: Recurse down subtracting each node's value from the target; succeed when a leaf is reached with the remaining amount exactly equal to that leaf's value.

JAVASCRIPT Copy

```
function hasPathSum(root,target){
  if(!root) return false;
  if(!root.left&&!root.right) return target===root.val;
  return hasPathSum(root.left,target-root.val)||hasPathSum(root.right,targ
}
```

23.104 Path Sum II

Approach: DFS while maintaining the current root-to-node path and running remainder, recording a copy of the path whenever a leaf reduces the remainder to zero, then backtrack.

JAVASCRIPT Copy

```
function pathSum(root,target){
  const res=[], path=[];
```

```
    if(!node.left&&!node.right&&rem===0) res.push([...path]);
    else { dfs(node.left,rem); dfs(node.right,rem); }
    path.pop();
  };
  dfs(root,target);
  return res;
}
```

23.105 Diameter of Binary Tree

Approach: Compute each node's height recursively while tracking the global maximum of left height plus right height, which is the longest path in edges through any node.

JAVASCRIPT Copy

```
function diameterOfBinaryTree(root){
  let best=0;
  const height=node=>{
    if(!node) return 0;
    const l=height(node.left), r=height(node.right);
    best=Math.max(best,l+r);
    return 1+Math.max(l,r);
  };
  height(root);
  return best;
}
```

23.106 Flatten Binary Tree to Linked List

Approach: Morris-style in-place rewiring: for each node with a left subtree, splice that subtree between the node and its right child by attaching the old right to the left subtree's rightmost node.

JAVASCRIPT Copy

```
function flatten(root){
  let curr=root;
```

```
      while(prev.right) prev=prev.right;
      prev.right=curr.right;
      curr.right=curr.left;
      curr.left=null;
    }
    curr=curr.right;
  }
}
```

23.107 Construct from Preorder and Inorder

Approach: Consume preorder left-to-right for roots and use an index map of inorder positions to split each range into left and right subtrees in O(n).

JAVASCRIPT Copy

```
function buildTree(preorder,inorder){
  const idx=new Map();
  inorder.forEach((v,i)=>idx.set(v,i));
  let pre=0;
  const build=(lo,hi)=>{
    if(lo>hi) return null;
    const val=preorder[pre++];
    const node={val,left:null,right:null};
    const mid=idx.get(val);
    node.left=build(lo,mid-1);
    node.right=build(mid+1,hi);
    return node;
  };
  return build(0,inorder.length-1);
}
```

23.108 Serialize and Deserialize

JAVASCRIPT Copy

```
  return `${root.val},${serialize(root.left)},${serialize(root.right)}`;
}
function deserialize(data) {
  const vals = data.split(',');
  let i = 0;
  function build() {
    if (vals[i] === 'null') { i++; return null; }
    const node = new TreeNode(Number(vals[i++]));
    node.left = build(); node.right = build();
    return node;
  }
  return build();
}
```

23.109 Binary Tree Maximum Path Sum

Approach: For each node compute the best downward gain from each child (clamped at zero) and update a global max with node value plus both gains, while returning value plus the single larger gain to the parent.

JAVASCRIPT Copy

```
function maxPathSum(root){
  let best=-Infinity;
  const gain=node=>{
    if(!node) return 0;
    const l=Math.max(0,gain(node.left));
    const r=Math.max(0,gain(node.right));
    best=Math.max(best,node.val+l+r);
    return node.val+Math.max(l,r);
  };
  gain(root);
  return best;
}
```

23.110 Count Complete Tree Nodes

Approach: Measure leftmost and rightmost spine depths; if equal the subtree is perfect so return 2^h - 1, otherwise recurse on both children, giving O(log^2 n).

JAVASCRIPT Copy

```
function countNodes(root){
  if(!root) return 0;
  const leftH=n⇒{ let h=0; while(n){h++;n=n.left;} return h; };
  const rightH=n⇒{ let h=0; while(n){h++;n=n.right;} return h; };
  const l=leftH(root), r=rightH(root);
  if(l===r) return (1<<l)-1;
  return 1+countNodes(root.left)+countNodes(root.right);
}
```

QUICK CHECK — no marks, just for you.

1. The lesson's iterative inorder traversal and recursive invertTree are run against the same tree, before and after inverting it. What is printed?

JAVASCRIPT Copy

```
function TreeNode(val) { this.val = val; this.left = null; this.right = null

function inorderTraversal(root) {
  const result = [], stack = [];
  let current = root;
  while (current || stack.length) {
    while (current) { stack.push(current); current = current.left; }
    current = stack.pop();
    result.push(current.val);
    current = current.right;
  }
  return result;
}

function invertTree(root) {
  if (!root) return null;
  [root.left, root.right] = [invertTree(root.right), invertTree(root.left)];
  return root;
}
```

```
root.left.left = new TreeNode(1);
root.left.right = new TreeNode(3);

console.log(inorderTraversal(root).join(','));
invertTree(root);
console.log(inorderTraversal(root).join(','));
```

- ○ 4,2,6,1,3
4,6,2,3,1
- ○ 1,2,3,4,6
1,2,3,4,6
- ○ 1,2,3,4,6
6,3,4,2,1
- ○ 1,2,3,4,6
6,4,3,2,1

Check answer

2. Following the lesson's approach, how is the Binary Tree Right Side View produced?

- ○ Run an inorder traversal and keep every node that has no left child
- ○ Run a preorder traversal and keep every node whose right pointer is null
- ○ Run a BFS level-order traversal and take the last element of each level
- ○ Recursively walk only the right child from the root, collecting every node reached

Check answer

3. In the Maximum Path Sum approach, each node contributes val + max(0, leftGain) + max(0, rightGain). Why are the child gains wrapped in max(0, ...)?

- So a subtree whose best path sum is negative is dropped rather than dragging the total down
- So the result is clamped to be non-negative, since path sums cannot be negative
- So each node is guaranteed to include at least one child in its path
- So the algorithm never reads a gain from a null child, which would otherwise be undefined

Check answer

Previous
Greedy

Next
Graphs

LEARN MORE

ARTICLES & DOCS

Binary search tree (VisuAlgo)
visualgo.net

VIDEOS



Binary tree traversals
YouTube



Intro to binary trees & traversal
YouTube



Tree traversal algorithms
YouTube